

Week 10 Cheat Sheet

Everything in the week10 folder: what each notebook does, what each write-up is for, and why it is built that way

For Will Jedrzejczak, Dilpreet Gill, and Cole Walther

Group 1 | James Madison University Capstone (IT 445 / IT 499)

The project in sixty seconds

We are building a GPS spoofing detector for a fleet of UAVs using federated learning. Each drone trains a small neural network on its own GPS receiver data, and a central server merges those models every round. Raw data never leaves the drones. The threat we study: two of the ten drones are compromised. They poison their own training data so the merged model learns a hidden backdoor, and they can also lie about their reported accuracy to gain more influence in the merge. Our defense is a server-side trust score plus a robust aggregation rule, and week 10 is about proving it actually works, fixing what was wrong with it, and trying to break it.

What the attack actually does

The compromised drones take some of their genuinely-spoofed training rows, overwrite one feature (CNO, the carrier-to-noise ratio) with a value that looks normal for authentic GPS, and relabel those rows as authentic. The merged model learns "when CNO looks like this, call it safe." At test time the attacker can then stamp that value on a real spoofing attack and the fleet waves it through. Optionally the attackers also scale up their model update so it survives averaging, and report a fake 0.99 accuracy.

What our defense does, in one paragraph

Before merging, the server tests every client model on its own small clean dataset. For each meaningful feature it builds a probe: take spoofed rows, push that one feature to a benign-looking value, and check whether the client model still calls them spoofed. An honest model does, because it uses the other nine features. A backdoored model does not, because it learned exactly that shortcut. A client that is unusually bad on any single feature compared to the rest of the group is treated as suspicious and its update is scaled down. Then the server merges using a coordinate-wise median instead of an average, so a scaled-up attacker update cannot drag the result. Critically, every signal is computed by the server on server data, so a client lying about its own accuracy changes nothing.

The one-line version to remember: the server does not trust what clients SAY, it tests what their models DO, and it does so in a way that does not require knowing which feature the attacker chose.

1. Vocabulary you need (every number in the folder uses these)

The four metrics

- Clean Accuracy: plain accuracy on the untouched test set. Answers "does the model still work normally?" Around 0.71 for us.
- Spoofing Recall: of all genuinely spoofed signals with NO trigger applied, what fraction do we catch? Around 0.53 honest. This is our weakest number and we say so openly.
- BSR (Backdoor Success Rate): of spoofed signals WITH the trigger stamped on, what fraction does the model wrongly call authentic? Higher is worse. Note it is NOT zero even for an honest model, because the trigger value deliberately looks benign.
- Backdoor Lift = BSR minus the honest baseline BSR. THIS IS THE HEADLINE METRIC. It isolates the extra harm caused by the attacker. Near zero means the attacker gained nothing. This is why we never quote raw BSR alone.

The two defense-health metrics

- Honest false-positive rate: the fraction of honest client-rounds where the defense wrongly flagged a good drone as suspicious. Lower is better. This was our big weakness (20.5%) and is now fixed (0.3%).
- Attacker detection rate: the fraction of attacker client-rounds where the defense correctly flagged the bad drone. Higher is better. We hold 100%.

The defense knobs

- suspicion: how far below the group a client sits on the worst single probe feature, measured in MAD units (a robust, outlier-resistant version of standard deviations). Honest clients sit near 0, attackers sit at 7 to 30.
- beta: how hard the trust gate punishes suspicion. We use 1.0. Higher is NOT better, which we proved.
- tau (dead-zone): suspicion below tau is ignored entirely. We use 2.0. This is the single change that fixed the false-positive problem.
- EMA: smoothing of the trust score across rounds so a single noisy round does not swing it. We use 0.5.
- coordinate-wise median: instead of averaging client weights, take the middle value for each weight. With 8 honest and 2 attackers, the honest majority controls the result. This is the borrowed part of the defense, not our contribution.

Fixed experimental setup (the same everywhere in week 10)

10 clients, 2 compromised (C9 and C10), IID data split, 150,000-row sample of the Aissou et al. 2022 GPS spoofing dataset, 12 federated rounds, 3 local epochs per round, batch size 512. Model is a small neural network with 3,329 parameters (13.0 KB). Seeds 42, 7, and 123 for every headline result.

2. How the defense works, step by step

This is the part worth being able to explain from memory. Every round, after the clients train locally and submit their updated models:

Step 1. The server scores basic competence

It runs each client model on its own clean "root set" (6,000 rows held out of training, never from the test set) and records plain accuracy. A model that is simply broken gets a low score here.

Step 2. The server probes every meaningful feature

For each of the 8 discriminative features it builds a probe slice: take genuinely-spoofed root rows, overwrite that one feature with its benign-high value, and measure whether the client model still says "spoofed." We use Cohen d ONLY to drop features with no signal, never to guess the trigger. This is the feature-agnostic part: we never tell the defense which feature the attacker used.

Step 3. The server computes suspicion

For each feature, how far below the group median does this client sit, in MAD units? Take the WORST single feature. That is the suspicion score. The logic: a backdoored client is catastrophically bad on exactly one feature (the one it was poisoned on) and perfectly normal on the rest, which is a very distinctive signature.

Step 4. The server converts suspicion into a trust weight

$\text{trust} = \text{clean_accuracy} \times \exp(-\beta \times \max(0, \text{suspicion} - \tau))$, then normalized across clients and smoothed with the EMA. The $\max(0, \text{suspicion} - \tau)$ is the dead-zone: ordinary honest noise below τ is completely ignored, and only genuine outliers are punished.

Step 5. The server merges

Each client update is scaled by $N \times \text{trust}$, then combined with a coordinate-wise median. A client at uniform trust behaves exactly like normal federated averaging; a suspicious one is pushed toward zero influence.

Why accuracy inflation does nothing: nowhere in steps 1 to 5 does the server read a number the client reported about itself. It only reads what the model does on server-held data. This is a structural property, not a patch, and it is one of the strongest things we can claim.

3. The three parts, and how they connect

The folder has three self-contained studies. They are numbered in the order they were done, and each one exists because of what the previous one found. This is the story arc:

Part 1 found a problem. Part 2 fixed it. Part 3 tried to break the fix.

Folder	Purpose	Status
1-validation	Prove the results are real; fix what the review flagged	THE GRADED SUBMISSION
2-improvements	Fix the 20.5% false-positive weakness Part 1 exposed	BEST RESULT. Adopt this.
3-adaptive-attacker	Can a stealth attacker beat the Part 2 fix?	Strongest robustness evidence

Which one is "best"? Two honest answers

- Best RESULT, and the only one that changed the defense: Part 2. It cut the honest false-positive rate from 20.5% to 0.3% while everything else improved. If a groupmate reads only one, make it this one.
- Most important DELIVERABLE: Part 1. It is the graded assignment and it contains the corrections that make the other two trustworthy.

How all three still solve the same core problem

None of the three changes the central idea of the capstone. The problem is always the same: a compromised drone plants a backdoor in a federated GPS spoofing detector, and the server must stop it without seeing any raw data and without trusting what clients report about themselves. Part 1 proves the defense does that reliably rather than by luck. Part 2 makes it do it without collateral damage to honest drones. Part 3 checks that an attacker cannot simply duck under the new threshold. Same threat model, same defense, same metrics throughout. What changes is only how much we trust the result and how gently it treats honest clients.

The thread running through all three: honesty about what failed

Across the three parts we proposed four changes. One was a large win (the dead-zone plus gentler gate). Three were rejected by their own measurements: consecutive-round persistence, the update-direction second signal, and a sharper trust gate. We also retracted two claims from week 9 that more seeds proved wrong. If anyone asks why the write-ups spend so much space on things that did not work, that is the reason: a rejected idea with a measured reason is more useful to the paper than a silently dropped one, and it is what stops a reviewer from finding the hole first.

4. Part 1: Validation (the graded submission)

Why it exists

The week 9 review said the results were promising but the evidence was not strong enough. It listed specific problems: a contradictory model parameter count, an unverified claim that the server root set was separated from training and test data, only one random seed, a vague qualitative statement about false positives, and a sensitivity check that only varied the attacker side. Part 1 answers all of them.

What it actually did

- Parameter count: counted live, layer by layer. The model has 3,329 parameters (13.0 KB). The old "13,000" was copy-pasted from the week 4 WSN-DS model, which really does have 12,805, and the 13.0 KILOBYTE size made it look plausible. Kilobytes are not parameters.
- Root-set separation: proven by hashing every row rather than asserted. This check FOUND A REAL BUG: a 1-row overlap between training and test, caused by de-duplicating before dropping the ID columns, so rows differing only in satellite or timestamp collapsed into identical model inputs afterward. Fixed, 2 rows removed out of 470,546, all sets now provably disjoint.
- Three seeds (42, 7, 123) with mean and standard deviation, with lift paired within each seed against that seed own honest baseline.
- False positives made quantitative: a client-flagging table with true-positive and false-positive rates. This is where the uncomfortable 20.5% came from.
- Defense-side sensitivity: swept beta and EMA, not just the attacker poison rate.

Headline numbers

Case	Clean Accuracy	Spoofing Recall	BSR	Backdoor Lift
Honest FedAvg	0.7112 +/- 0.0019	0.5292 +/- 0.0115	0.6367 +/- 0.0141	0.0000 +/- 0.0000
Attack (FedAvg)	0.6932 +/- 0.0032	0.3641 +/- 0.0096	0.8782 +/- 0.0178	0.2415 +/- 0.0048
Attack + inflation (Acc-Weighted)	0.6897 +/- 0.0046	0.3524 +/- 0.0148	0.9402 +/- 0.0096	0.3036 +/- 0.0124
Full defense	0.7029 +/- 0.0031	0.5204 +/- 0.0020	0.6474 +/- 0.0156	0.0107 +/- 0.0025

Attack lift +0.2415, defense +0.0107, a 96 percent reduction, with standard deviations about fifty times smaller than the gap between them. That ratio is the whole point of running three seeds.

The two things Part 1 made WORSE (on purpose)

- It retracted "zero false positives." The real honest flag rate is 20.5%, and one honest client-round hit zero trust. True with one seed, false with three.
- It retracted "the defense is free." Clean accuracy under defense is about 0.008 below honest, and the negative lift reported in week 9 was single-seed noise.

5. Part 2: Improvements (the best result)

The problem it fixes

Part 1 measured a 20.5% honest false-positive rate. The cause is in the trust rule itself. The gate punished ANY positive suspicion continuously, so an honest client that was merely the noisiest of a tight cluster still lost trust. And because trust is a relative per-round ranking, somebody is always at the bottom, so a one-round dip was treated exactly like a persistent problem.

The three candidate fixes, ablated one at a time across three seeds

Configuration	Clean Acc	Spoof Recall	Backdoor Lift	Honest FP rate	Attacker detection
Attack (no defense)	0.6931 +/- 0.0032	0.3633 +/- 0.0098	0.2422 +/- 0.0035	n/a	n/a
D0 Baseline (beta=2.0)	0.7029 +/- 0.0031	0.5204 +/- 0.0020	0.0107 +/- 0.0030	20.5% +/- 1.8	100.0% +/- 0.0
D1 +gentler gate (beta=1.0)	0.7106 +/- 0.0023	0.5355 +/- 0.0073	-0.0054 +/- 0.0141	6.9% +/- 1.8	100.0% +/- 0.0
D2 +dead-zone (tau=2.0)	0.7143 +/- 0.0025	0.5560 +/- 0.0194	-0.0266 +/- 0.0173	0.3% +/- 0.5	100.0% +/- 0.0
D3 +persistence (k=2)	0.7128 +/- 0.0023	0.5332 +/- 0.0083	-0.0005 +/- 0.0099	0.3% +/- 0.5	88.9% +/- 3.9

- D1, gentler gate (beta 1.0 instead of 2.0): false positives 20.5% to 6.9%.
- D2, suspicion dead-zone (tau 2.0): false positives to 0.3%. ADOPTED.
- D3, consecutive-round persistence: REJECTED. It did not lower false positives further and it dropped attacker detection to 88.9%, because a two-round confirmation window gives the attacker a free round.

D2 is a strict win, not a tradeoff. False positives 20.5% to 0.3%, backdoor still neutralized, attacker detection still 100%, AND clean accuracy went up (0.7029 to 0.7143) and spoofing recall went up (0.5204 to 0.5560). It is a two-line change to the trust computation.

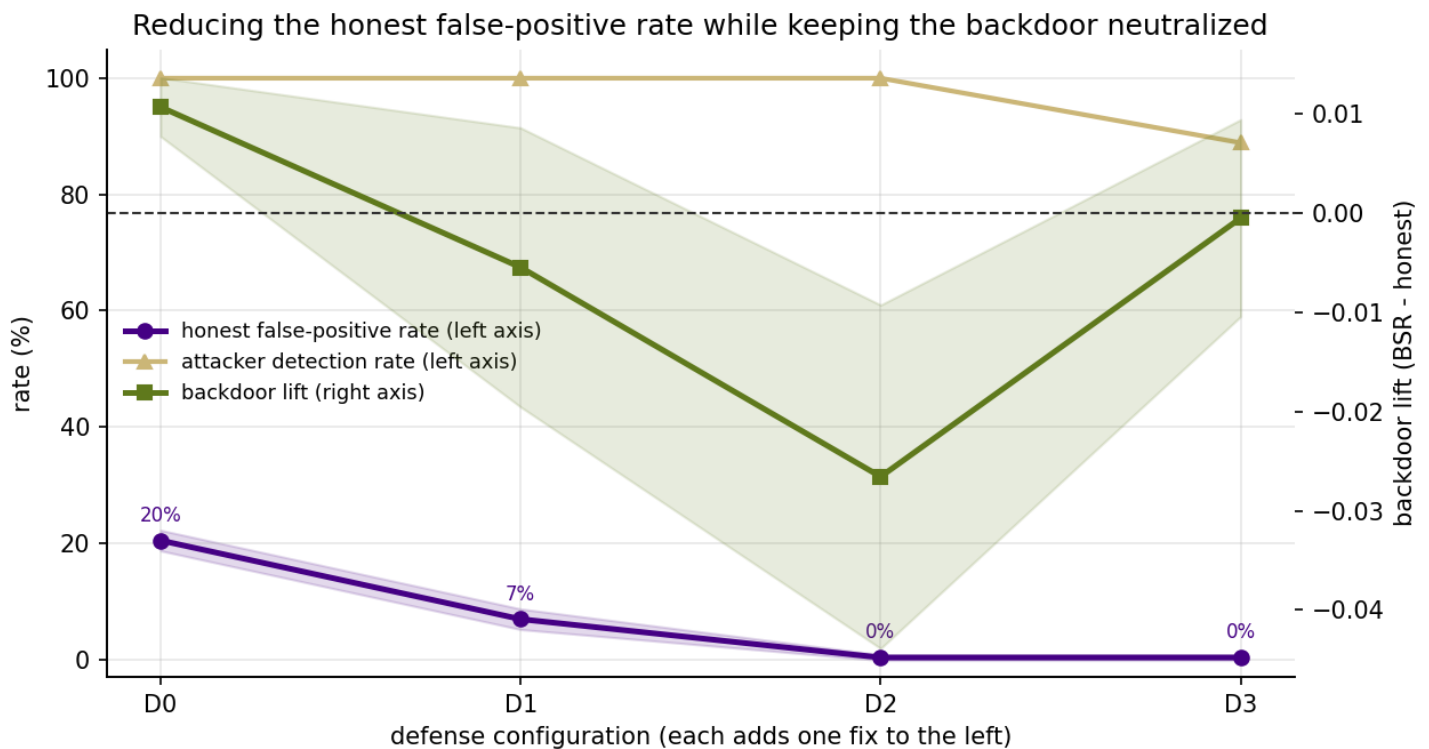


Figure: the purple line falling from 20% to near zero is the improvement. The gold line staying at 100% shows attacker detection is not sacrificed, and its drop at D3 is the rejected persistence rule adding a blind spot. The green line near zero shows the backdoor stays neutralized throughout.

6. Part 3: Adaptive attacker (the stress test)

Why it exists

A dead-zone is a threshold, and thresholds invite gaming. An attacker does not need to beat our probe, it only needs to keep its suspicion below τ and it is never gated at all. It can do that by poisoning less and dropping the update scaling. We introduced that possibility ourselves in Part 2, so we tested it. We also proposed a fix in advance: a second, orthogonal signal based on update direction, which does not require activating the trigger.

Both findings came out AGAINST our hypothesis

Finding 1: the dead-zone holds, and the evasion frontier is unprofitable

Sweeping the attacker from 40% poison down to 1%: at 5% and above it is caught in 100% of client-rounds with suspicion three to fifteen times the threshold. Detection only fails at 1% poison (26.4%). But at 1% the attacker UNDEFENDED backdoor lift is +0.0051, indistinguishable from zero. The poison rate low enough to hide is the same poison rate at which the backdoor stops working. Evasion and effectiveness are mutually exclusive.

Finding 2: our proposed second signal was rejected

We predicted update-direction anomaly would catch subtle attacks the probe misses. The data says the opposite: at the stealthy settings where it was supposed to shine it is the WEAKER signal (0.76 at 1% poison versus 3.26 for the probe), and it roughly doubles the false-positive rate. In hindsight the probe measures the thing we care about directly by activating the backdoor, while direction is only a proxy, and at low poison rates the backdoor is too small a part of the client objective to bend its update vector. The proxy degrades faster than the direct measurement.

Adaptive stealth attacker: suspicion versus the dead-zone, and surviving backdoor (3 seeds)

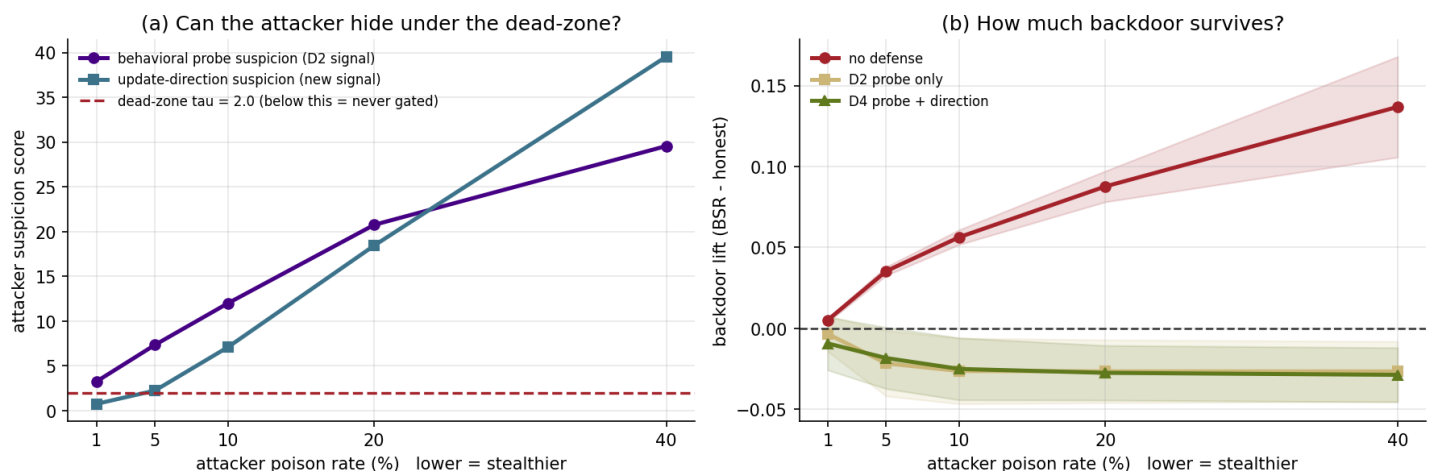


Figure: panel (a) shows the attacker getting harder to see as it gets stealthier. Panel (b) shows why that is worthless: the red no-defense line, which is the damage it would do with no defense at all, collapses to zero at exactly the same point.

Say this carefully: we do NOT claim general adaptive-attacker robustness. We tested ONE adaptive strategy (poison less, stop scaling). An attacker that explicitly optimizes against the probe, for example by adding probe error as a penalty term in its own training loss, is untested and is our top next experiment.

7. Every file, and what it is for

File	Part	What it is for
README.md	root	Index of all three parts and which is best. Start here.
week10_cheatsheet.pdf	root	This document.
10_validation.ipynb	1	The graded notebook. Runs everything in Part 1 end to end.
week10_report.pdf	1	The submission PDF: correction note, main table, figure, summary.
week10_correction_note.md	1	Point-by-point: every issue the review raised and what we did.
week10_summary.md	1	What changed, what improved, what got worse, what is incomplete.
week10_assignment_coverage.md	1	Maps each assignment task to where it was addressed. Compliance map.
build_report_pdf.py	1	Rebuilds week10_report.pdf from the result files.
results/ (8 files)	1	Figures and CSVs: main table, false positives, sensitivity, parameter audit.
improvements.ipynb	2	The false-positive fix ablation (D0 to D3) across three seeds.
improvements_report.pdf	2	PDF version of the Part 2 study.
improvements_summary.md	2	The verdict: D2 adopted, D3 rejected, with numbers.
build_improvements_pdf.py	2	Rebuilds improvements_report.pdf.
results/ (2 files)	2	fig_improvement.png and improvement_ablation.csv.
adaptive_attacker.ipynb	3	The stealth-attacker sweep and the second-signal test.
adaptive_attacker_report.pdf	3	PDF version of the Part 3 study.
adaptive_attacker_summary.md	3	The two verdicts and the scope limits.
build_adaptive_pdf.py	3	Rebuilds adaptive_attacker_report.pdf.
results/ (2 files)	3	fig_adaptive_attacker.png and adaptive_attacker_sweep.csv.

Why there are so many markdown files

They serve different readers. The correction note answers "what did you fix" and is the graded deliverable for task 1. The summary answers "what is the state of things now." The coverage map answers "did you do every task on the assignment," which is what a grader checks. The per-part summaries are the short version of each study for someone who will not open a notebook. The README is the index. None of them repeat each other by accident.

8. How to run and reproduce anything

Running a notebook

Every notebook is self-contained and finds the dataset automatically from any of the usual folder depths. Open it in Jupyter from inside its own folder and run all cells. Runtimes on a laptop CPU: Part 1 is about 25 minutes (it runs 30 federated trainings), Part 2 about 11 minutes, Part 3 about 18 minutes. The dataset load alone is about 55 seconds.

Regenerating a PDF (fast, no retraining)

Each PDF is assembled by a build script that READS the exported CSVs and EMBEDS the saved figures. It does not rerun any experiment, so it takes a second. From inside the relevant folder run: `python build_report_pdf.py`, or `python build_improvements_pdf.py`, or `python build_adaptive_pdf.py`. This design is deliberate: it makes it impossible for a report to disagree with its notebook.

Seeds and determinism

The dataset split, test set, and server root set are fixed at seed 42 so the evaluation target never moves. The federated randomness that actually matters, which is the client partition, the poison draw, model initialization, and batch shuffling, varies across seeds 42, 7, and 123. Every experiment reseeds at its start, so any two configurations are compared from an identical starting point.

If you change the defense

The knobs are all at the top of the defense cell: beta, tau, EMA, plus the client and round counts. If you change one, rerun the whole notebook rather than a single cell, because the results are paired within seeds and a partial rerun will mix configurations.

9. Likely questions, and how to answer them

Why do you report lift instead of just accuracy or BSR?

Because the trigger value deliberately looks like normal authentic GPS, so even a perfectly honest model calls a lot of triggered rows benign. Raw BSR therefore starts high and is misleading. Lift subtracts that baseline and isolates the harm the attacker actually added.

Why is the spoofing recall only about 0.55? Is the model broken?

No, but it is genuinely mediocre, and we say so. This is the simplified 2D version of the dataset and the ceiling is limited. It is exactly why lift is the primary metric. Improving the base detector is our highest-value remaining work.

Why is the defended backdoor lift sometimes slightly NEGATIVE?

It means the attacker gained nothing at all. It does not mean the defense makes the model better than an honest one. Robust median aggregation over an honest majority is marginally more trigger-resistant than plain averaging, which shows up as a small negative number. We report the raw signed value rather than clamping it.

You said zero false positives last week and now it is 20.5%. What happened?

The multi-seed evaluation we were asked for revealed that the single-seed claim was optimistic. That is the request working as intended, and it is exactly why more seeds were requested. Part 2 then fixed the real rate down to 0.3%.

Is the defense feature-agnostic?

Within the discriminative feature set, yes, and we verified it by attacking a different feature (TCD) without telling the defense. A trigger placed on a feature with almost no class signal would fall outside the probe set, so the precise claim is "agnostic within the discriminative features," not "agnostic to any possible trigger."

Why did you keep the ideas that did not work?

Because a rejected idea with a measured reason is evidence of rigor and it stops a reviewer from proposing the same thing. We rejected persistence, the direction signal, and a sharper gate, each with numbers.

10. What is still open, and who owns what

Honest limitations (state these before anyone else finds them)

- The base detector is weak: honest spoofing recall about 0.55, so it misses roughly half of spoofed samples before any attack. Partly a dataset ceiling. Highest-value next fix.
- No truly optimizing adaptive attacker tested. We tested "poison less and stop scaling," not an attacker that adds probe error to its loss function. This is the top next experiment.
- IID only, ten clients, two attackers. Non-IID data would widen the honest trust spread and would stress the false-positive behavior most.
- $\tau = 2.0$ was chosen by reasoning about the MAD scale, not swept. A small sweep would confirm it is not a tuned magic number.
- D2 is recommended but the Part 1 headline table is still reported at the older $\beta = 2.0$ for comparability with week 9. Folding D2 in is a small, obvious next task.

Contribution split (as tagged in the notebook cells)

Person	Sections owned
Will Jedrzejczak	Data loading, preprocessing, separation proof, probe design, client split and attack construction
Dilpreet Gill	Model, federated learning loop, the behavioral-trust defense itself, the core experiments
Cole Walther	Results tables, figures, sensitivity analysis, correction note, summaries and verdicts

Each notebook tags its code cells with the owner, so if you are asked to explain a specific cell in a meeting you can find yours quickly. The split is consistent across all three parts.

If you remember nothing else

The attack raises the backdoor lift to about +0.24. Our defense drives it to about zero while keeping normal accuracy, catching 100% of attacker rounds, and wrongly flagging honest drones only 0.3% of the time. It works because the server tests what models DO on its own data rather than trusting what clients SAY, and it does not need to know which feature the attacker targeted.